

# Day 1 Lab Notes

# Unit Testing with XUnit v3 & FluentAssertions

## Prerequisites

Before the lab, verify your environment is ready:

| Requirement          | Version | Check                         |
|----------------------|---------|-------------------------------|
| .NET SDK             | 9.0+    | <code>dotnet --version</code> |
| Visual Studio 2022   | 17.8+   | Help → About                  |
| VS Code + C# Dev Kit | latest  | Extensions panel              |
| Git (optional)       | any     | <code>git --version</code>    |

If multiple SDKs are installed, confirm .NET 9 is active:

```
dotnet --list-sdks
```

To pin the version for this folder, run `dotnet new globaljson --sdk-version 9.0.xxx`

## Lab Folder Structure

Day1\Solutions\Lab\

└─ Day1.Lab.sln

```
|— Day1.Lab.BankAccount\
```

```
|   └─ Day1.Lab.BankAccount.csproj    ← class library (already
implemented)
```

└─ BankAccount.cs ← System Under Test – DO NOT EDIT

```
└─ Day1.Lab.BankAccount.Tests\
```

|— Day1.Lab.BankAccount.Tests.csproj

└─ BankAccountTests.cs ←  YOUR WORK IS HERE

# Setup Steps

## Step 1 – Open the Solution

### Visual Studio 2022:

File → Open → Project/Solution → Day1.Lab.sln

### VS Code:

```
cd "Day1\Solutions\Lab"  
code .
```

### Terminal only:

```
cd "Day1\Solutions\Lab"
```

---

## Step 2 – Restore NuGet Packages

```
dotnet restore
```

### Expected output:

Restored Day1.Lab.BankAccount\Day1.Lab.BankAccount.csproj

Restored Day1.Lab.BankAccount.Tests\Day1.Lab.BankAccount.Tests.csproj

### Packages included in the project – no manual install needed:

- `xunit.v3` – test framework
  - `xunit.runner.visualstudio` – Test Explorer integration
  - `FluentAssertions` – readable assertions
  - `Microsoft.NET.Test.Sdk` – test host
- 

## Step 3 – Build the Solution

```
dotnet build
```

Expected: Build succeeded. 0 Error(s)

If you see errors in `BankAccount.cs`, you may have accidentally edited it.  
Restore it from the course materials zip.

---

## Step 4 – Run the Tests (They Will Fail)

```
dotnet test
```

You will see `NotImplementedException` for every test – this is **expected**.

Your job is to replace each `throw new NotImplementedException()` with real assertions.

```
# Verbose output – see exactly which tests failed and why
dotnet test --logger "console;verbosity=detailed"
```

---

## Step 5 – Open `BankAccountTests.cs`

Every test method follows this structure:

```
[Fact]
public void Deposit_ValidAmount_IncreasesBalance()
{
    // Arrange – inputs are already set up for you
    var account = new BankAccount("Alice", initialBalance: 100m);

    // Act – the action is already called for you
    account.Deposit(50m);

    // TODO: Assert – this is what you write
    // account.GetBalance().Should().Be(???);
    throw new NotImplementedException();
}
```

**Your task:** Remove the `throw new NotImplementedException()` and write the assertion.

---

## FluentAssertions Quick Reference

---

| Scenario               | FluentAssertions Syntax                                   |
|------------------------|-----------------------------------------------------------|
| Value equals expected  | <code>result.Should().Be(expected)</code>                 |
| Value is null          | <code>result.Should().BeNull()</code>                     |
| Value is not null      | <code>result.Should().NotBeNull()</code>                  |
| Boolean is true        | <code>flag.Should().BeTrue()</code>                       |
| Boolean is false       | <code>flag.Should().BeFalse()</code>                      |
| Decimal is positive    | <code>amount.Should().BePositive()</code>                 |
| Greater than           | <code>x.Should().BeGreaterThan(0)</code>                  |
| Exception thrown       | <code>act.Should().Throw&lt;ExceptionType&gt;()</code>    |
| Exception with message | <code>.Throw&lt;T&gt;().WithMessage("*keyword*")</code>   |
| Collection size        | <code>list.Should().HaveCount(3)</code>                   |
| Collection contains    | <code>list.Should().Contain(x =&gt; x.Name == "A")</code> |

## Lab Exercises

Complete each test method in `BankAccountTests.cs`.

### Exercise 1 – Deposit (3 tests)

| # | Test Method                                                            | What to Assert                           |
|---|------------------------------------------------------------------------|------------------------------------------|
| 1 | <code>Deposit_ValidAmount_IncreasesBalance</code>                      | Balance = initial + deposited            |
| 2 | <code>Deposit_MultipleDeposits_AccumulatesCorrectly</code>             | Balance = sum of all deposits            |
| 3 | <code>Deposit_ZeroOrNegativeAmount_ThrowsArgumentEx<br/>ception</code> | <code>ArgumentException</code> is thrown |

**Hint for test 3:**

```

Action act = () => account.Deposit(invalidAmount);
act.Should().Throw<ArgumentException>();

```

### Exercise 2 – Withdraw (4 tests)

| # | Test Method                                                           | What to Assert                                                           |
|---|-----------------------------------------------------------------------|--------------------------------------------------------------------------|
| 4 | <code>Withdraw_ValidAmount_DecreasesBalance</code>                    | Balance = initial - withdrawn                                            |
| 5 | <code>Withdraw_ExactBalance_LeavesZeroBalance</code>                  | Balance = 0                                                              |
| 6 | <code>Withdraw_MoreThanBalance_ThrowsInvalidOperationException</code> | <code>InvalidOperationException</code> + message contains "Insufficient" |
| 7 | <code>Withdraw_ZeroOrNegativeAmount_ThrowsArgumentException</code>    | <code>ArgumentException</code> is thrown                                 |

#### Hint for test 6:

```

Action act = () => account.Withdraw(100m);
act.Should()
    .Throw<InvalidOperationException>()
    .WithMessage("*Insufficient*");

```

## Exercise 3 – Transfer (3 tests)

| #  | Test Method                                                              | What to Assert                               |
|----|--------------------------------------------------------------------------|----------------------------------------------|
| 8  | <code>Transfer_ValidAmount_MovesMoneyBetweenAccounts</code>              | Source decreases AND destination increases   |
| 9  | <code>Transfer_InsufficientFunds_ThrowsAndLeavesAccountsUnchanged</code> | Exception thrown + both balances unchanged   |
| 10 | <code>Transfer_NullDestination_ThrowsArgumentNullException</code>        | <code>ArgumentNullException</code> is thrown |

#### Hint for test 8 (asserting two values):

```

source.GetBalance().Should().Be(150m);
destination.GetBalance().Should().Be(250m);

```

## Exercise 4 – Constructor Validation (2 tests)

| #  | Test Method                                                             | What to Assert                        |
|----|-------------------------------------------------------------------------|---------------------------------------|
| 11 | <code>Constructor_NegativeInitialBalance_ThrowsArgumentException</code> | <code>ArgumentException</code> thrown |

| #  | Test Method                                          | What to Assert                             |
|----|------------------------------------------------------|--------------------------------------------|
| 12 | Constructor InvalidOwnerName_ThrowsArgumentException | ArgumentException thrown for "", " ", null |

Hint for test 12 ( [Theory] ):

```
Action act = () => new BankAccount(invalidName!, 100m);
act.Should().Throw<ArgumentException>();
```

---

## Running Tests

### CLI

```
# Run all tests
dotnet test

# Run a specific test by name
dotnet test --filter "Deposit_ValidAmount_IncreasesBalance"

# Run all tests in a category
dotnet test --filter "FullyQualifiedName~BankAccountTests"

# Verbose output
dotnet test --logger "console;verbosity=detailed"
```

### Visual Studio Test Explorer

1. **View → Test Explorer** (or `Ctrl+E, T`)
2. Click **Run All** (or the green play button)
3. Red = failing, Green = passing, Yellow = skipped

### VS Code

1. Install **C# Dev Kit** extension
  2. Click the **flask icon** in the Activity Bar
  3. Click **Run All Tests**
-

# Expected Final Result

When all exercises are complete:

```
Total tests: 12
    Passed: 12
    Failed: 0
```

Every test method name tells you exactly what it verified. If a test fails, the FluentAssertions error message shows you the actual value vs. the expected value.

## Troubleshooting

| Problem                              | Solution                                            |
|--------------------------------------|-----------------------------------------------------|
| <code>NotImplementedException</code> | You haven't replaced the <code>throw</code> yet     |
| <code>Package not found</code>       | Run <code>dotnet restore</code>                     |
| <code>CS0246: type not found</code>  | Add <code>using FluentAssertions;</code> at the top |
| Test not appearing in Explorer       | Rebuild: <code>dotnet build</code>                  |
| Wrong balance after transfer         | Check source AND destination assertions             |

## What's Next – Day 2 Preview

| Topic            | What You'll Learn                             |
|------------------|-----------------------------------------------|
| Moq              | Replace real dependencies with test doubles   |
| Mock, Stub, Fake | The difference between test double types      |
| Setup & Verify   | Control mock behavior and assert interactions |
| API Unit Testing | Test controllers without starting a server    |
| Test Coverage    | Measure how much code your tests exercise     |